Name:K.SAI SNEHITH

HallTicket:2303A52006

Assginment:11.1

Batch:44

Task:01

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code:

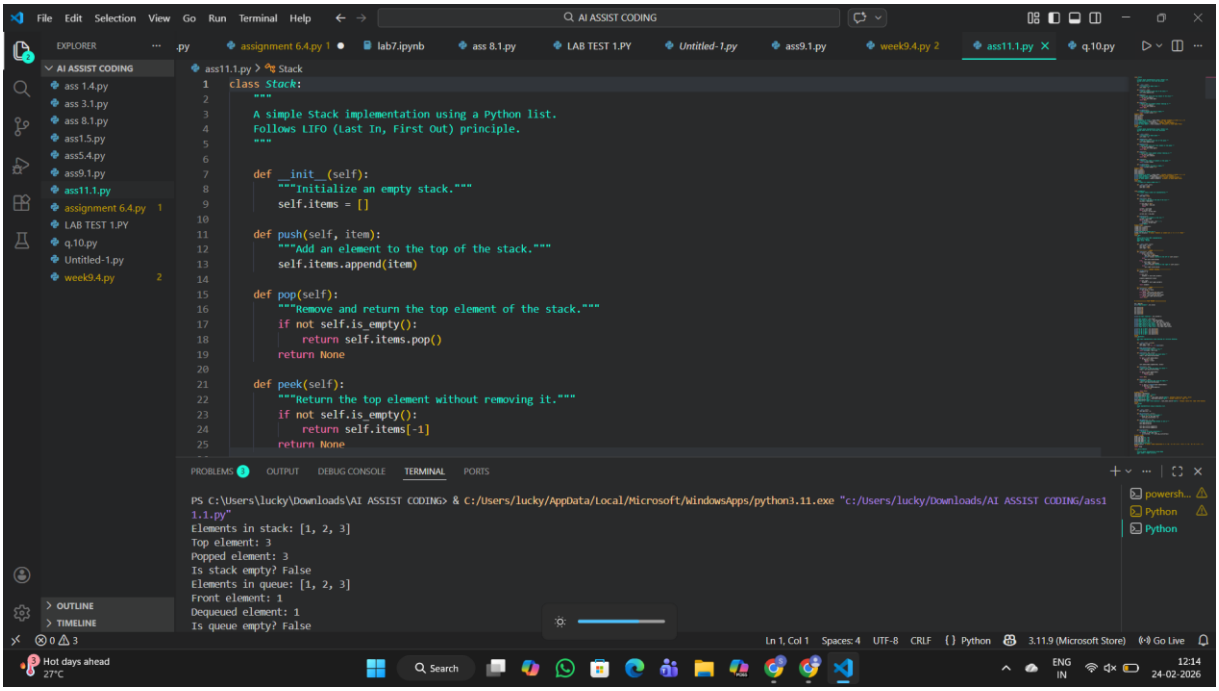
class Stack:

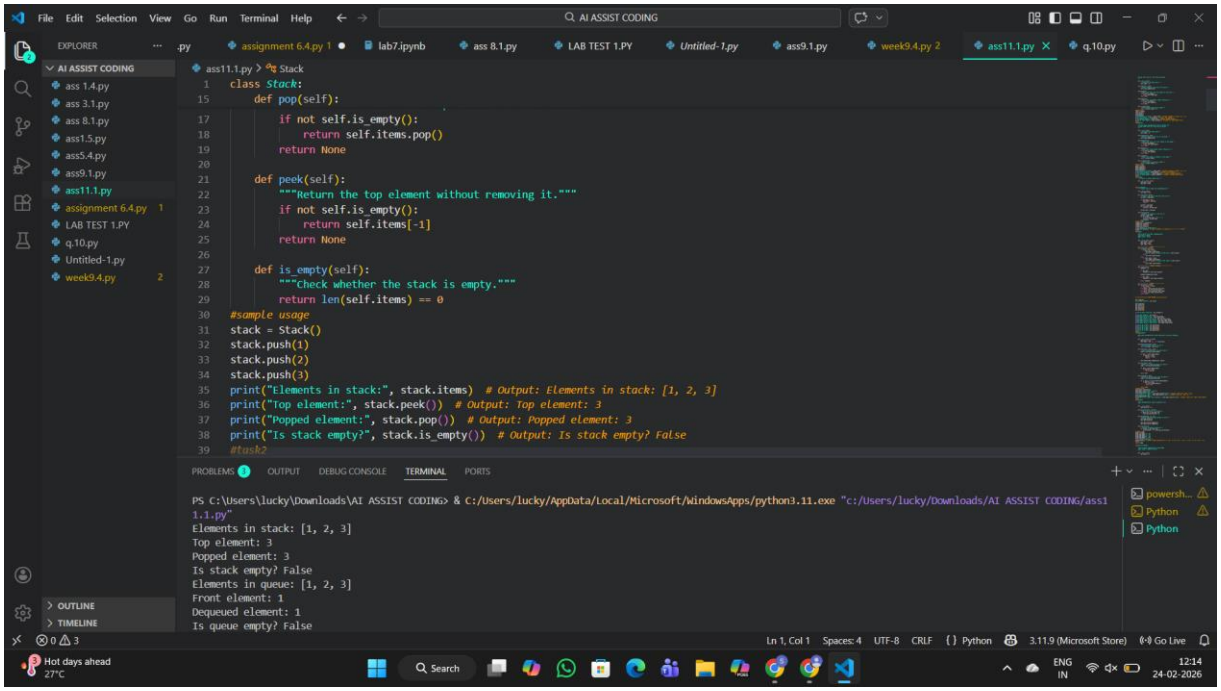
pass

Expected Output:

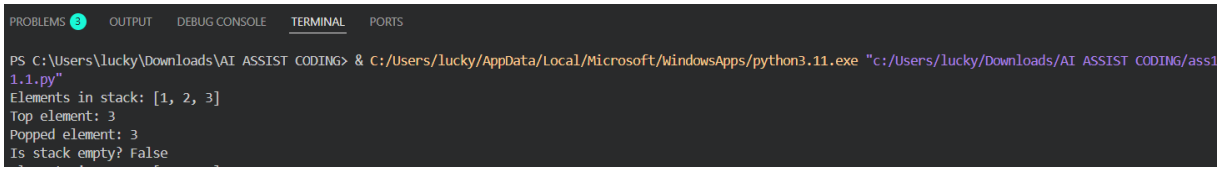
• A functional stack implementation with all required methods and docstrings.

Prompt: create stack for pop push and is empty function with dotstrings Code:





Output:



Observation:

The stack follows the LIFO (Last In, First Out) principle where the last inserted element is removed first.

push adds elements, pop removes the top element, and peek shows the top without removing it.

is_empty checks whether the stack contains any elements to prevent underflow errors Task:02

ask Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

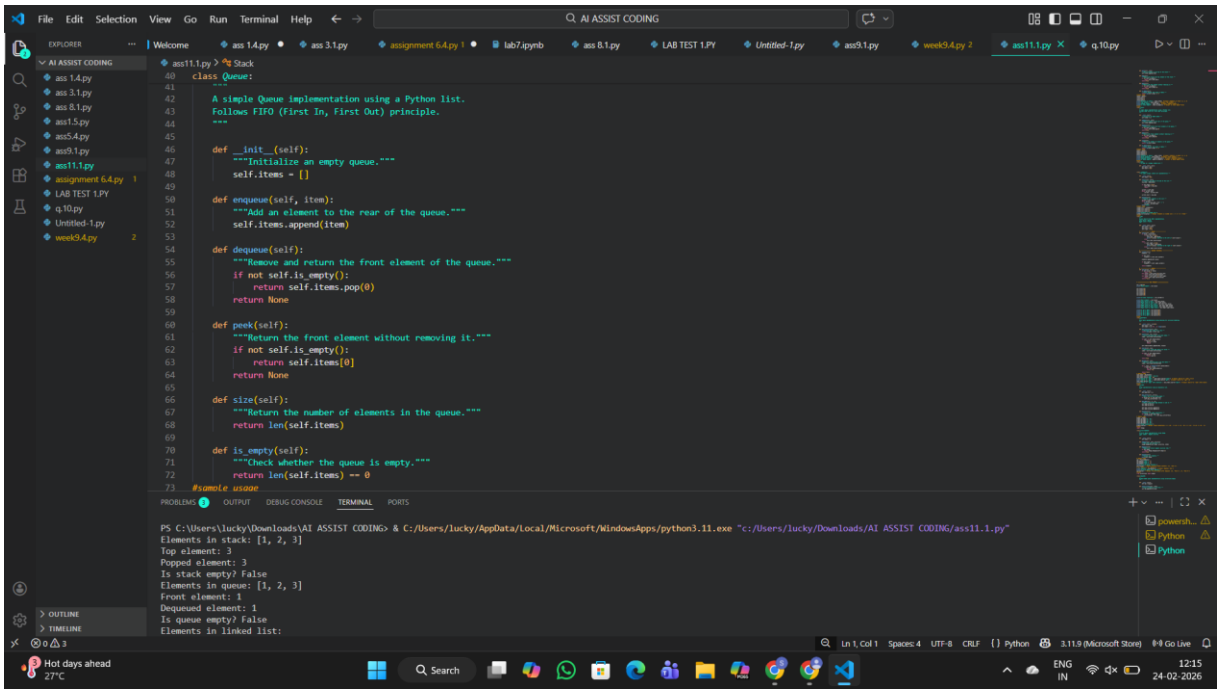
pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size

Method

Prompt:implement Quees following the fifo deque and peek Code:



Output:

```
Is stack empty? False
Elements in queue: [1, 2, 3]
Front element: 1
Dequeued element: 1
Is queue empty? False
```

Observation:

The queue follows the FIFO (First In, First Out) principle, where the first inserted element is removed first.

enqueue adds elements to the rear, while dequeue removes elements from the front of the queue.

peek shows the front element without removing it, and size returns the current number of elements in the queue. Task:03

ask: Use AI to generate a Singly Linked List with insert and display

methods. Sample Input Code: class Node:

pass class

LinkedList:

pass

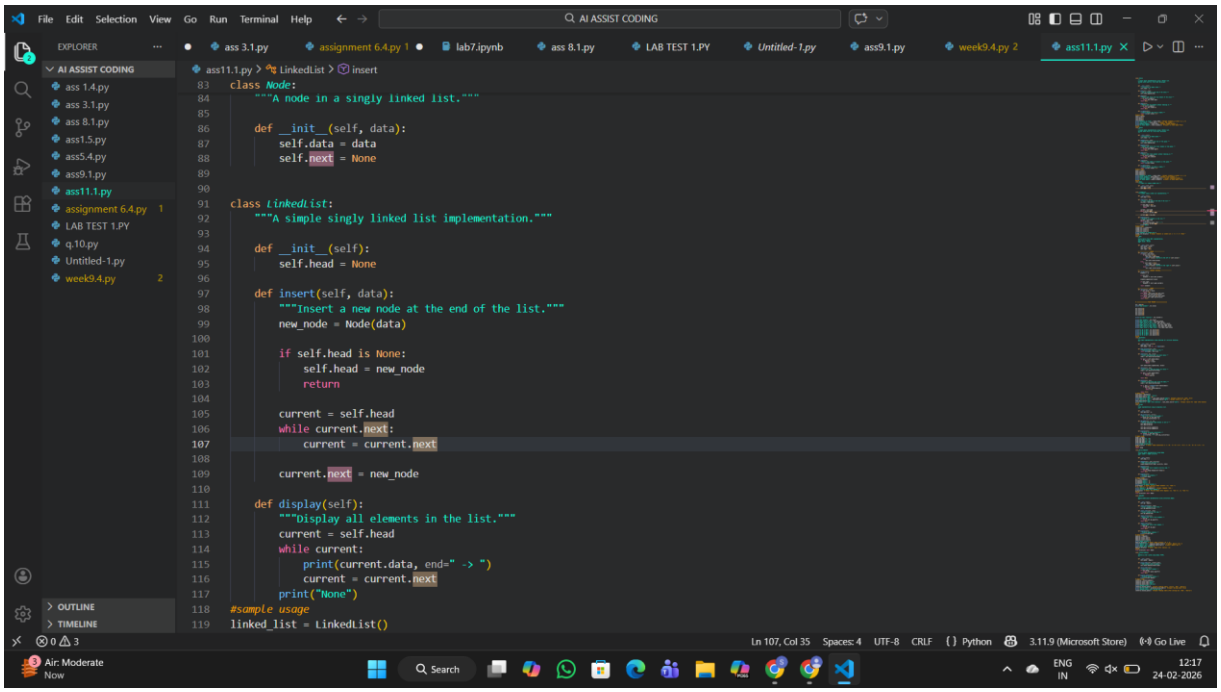
Expected Output:

• A working linked list implementation with clear method

Documentation

Prompt: create singleylinked list **Code**

:



```python
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    """A simple singly linked list implementation."""

    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node at the end of the list."""
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next

        current.next = new_node

    def display(self):
        """Display all elements in the list."""
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
#sample usage
linked_list = LinkedList()
```

Output:

```
Elements in linked list:
1 -> 2 -> 3 -> None
```

Observation:

1. A singly linked list stores elements in nodes where each node points to the next node in sequence.
2. The insert method adds new elements at the end by traversing the list until the last node.
3. The display method prints all elements sequentially from the head to the last node. Task:04 ask: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

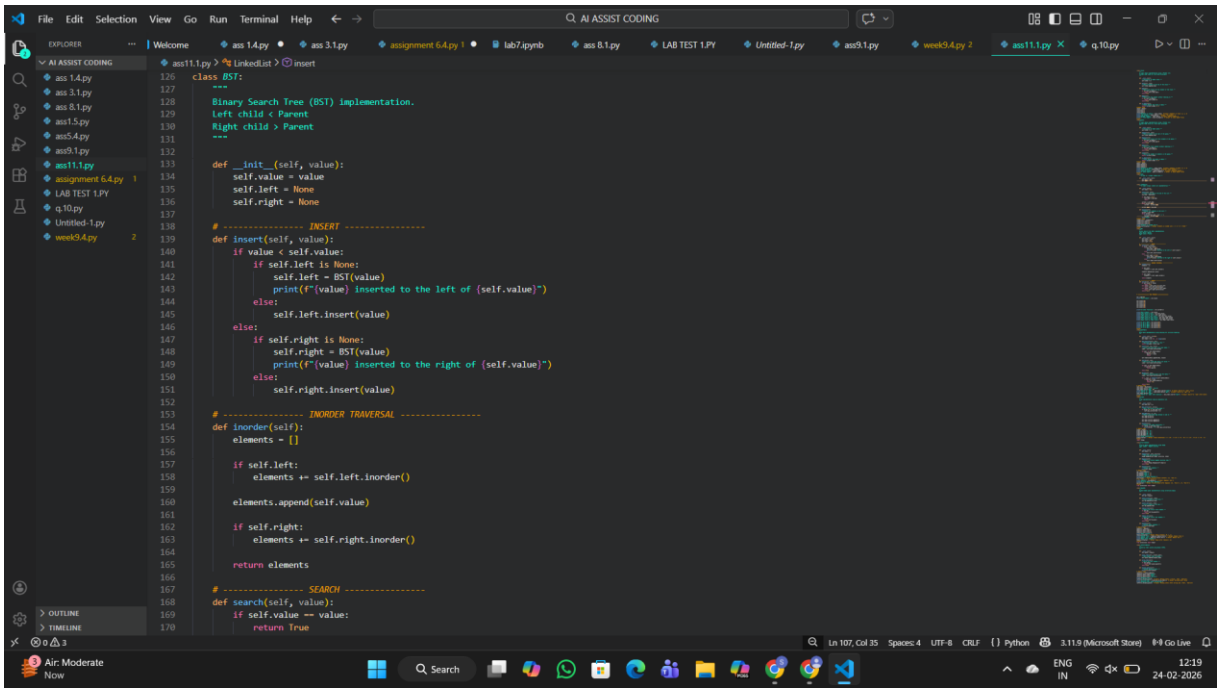
pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

Prompt: create a Binarysearch tree for the following reuirements

Code:



```python
class BST:
    """
    Binary Search Tree (BST) implementation.
    Left child < Parent
    Right child > Parent
    """

    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

    # ---------------- INSERT ----------------
    def insert(self, value):
        if value < self.value:
            if self.left is None:
                self.left = BST(value)
                print(f"{value} inserted to the left of {self.value}")
            else:
                self.left.insert(value)
        else:
            if self.right is None:
                self.right = BST(value)
                print(f"{value} inserted to the right of {self.value}")
            else:
                self.right.insert(value)

    # ---------------- INORDER TRAVERSAL ----------------
    def inorder(self):
        elements = []

        if self.left:
            elements += self.left.inorder()

        elements.append(self.value)

        if self.right:
            elements += self.right.inorder()

        return elements

    # ---------------- SEARCH ----------------
    def search(self, value):
        if self.value == value:
            return True
```

Output:

```
Head element: 50
30 inserted to the left of 50
70 inserted to the right of 50
20 inserted to the left of 30
40 inserted to the right of 30
60 inserted to the left of 70
80 inserted to the right of 70
In-order Traversal: [20, 30, 40, 50, 60, 70, 8
Root element: 50
Left child of root: 30
Right child of root: 70
Left child of left child: 20
Right child of left child: 40
Left child of right child: 60
Right child of right child: 80
Is 20 in BST? True
Is 90 in BST? False
Is 50 in BST? True
Is 30 in BST? True
Is 70 in BST? True
```

Observation:

A Binary Search Tree (BST) organizes data such that left child nodes contain smaller values and right child nodes contain larger values than the parent.

The recursive insert method places each new value in its correct sorted position while maintaining BST properties.

In-order traversal visits nodes in Left → Root → Right order, producing the elements in sorted (ascending) order. Task:05

ask Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and

delete methods. Sample Input Code: class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods

Prompt: implement hash Table in python Code:

```python
class HashTable:
    """
    Hash Table implementation using chaining for collision handling.
    """

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        """Generate hash index for a key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert key-value pair into hash table."""
        index = self.hash_function(key)

        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return

        self.table[index].append([key, value])

    def search(self, key):
        """Search for a key and return its value."""
        index = self.hash_function(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None

    def delete(self, key):
        """Delete a key-value pair from the table."""
        index = self.hash_function(key)
```

Output:

```
Search for 'name': Alice
Search for 'age': 30
Search for 'name' after deletion: None
```

Observation:

A hash table stores data using a hash function that maps keys to specific indices for fast access.

Collision handling with chaining uses linked lists (or lists) at each index to store multiple elements that hash to the same position.

The insert, search, and delete methods operate efficiently, typically in constant time on average

Task:06

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display

connections.

Prompt: create graphs my function find all the vertices add edges Code:

```python
print("Search for `name` after deletion:", hash_table.search("name"))  # Output: Search for `name` after deletion: None
#task6
class Graph:
    """
    Graph implementation using an adjacency list.
    """

    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        """Add a new vertex to the graph."""
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, v1, v2):
        """Add an undirected edge between v1 and v2."""
        self.add_vertex(v1)
        self.add_vertex(v2)

        self.adj_list[v1].append(v2)
        self.adj_list[v2].append(v1)

    def display(self):
        """Display the graph connections."""
        for vertex in self.adj_list:
            print(vertex, "->", self.adj_list[vertex])
# Sample usage
graph = Graph()
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "D")
graph.add_edge("C", "D")
print("Graph connections:")
graph.display()  # Output: Graph connections: A -> ['B', 'C'] B -> ['A', 'D'] C -> ['A', 'D'] D -> ['B', 'C']
#task7
import heapq
```

Output:

```
Graph connections:
A -> ['B', 'C']
B -> ['A', 'D']
C -> ['A', 'D']
D -> ['B', 'C']
```

Observation:

An adjacency list represents a graph efficiently by storing each vertex and its connected neighbors.

Adding an edge updates the connection list of both vertices in an undirected graph.

This representation is memory-efficient for sparse graphs compared to

adjacency matrices Task07:

ask: Use AI to implement a priority queue using Python’s heapq module.

Sample Input Code: class

PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Prompt: for this implement the priority Quee for following requirements Code:

```python
#task7
import heapq

class PriorityQueue:
    """
    Priority Queue implementation using heapq.
    Lower number = Higher priority.
    """

    def __init__(self):
        self.heap = []

    def enqueue(self, item, priority):
        """Add item with priority."""
        heapq.heappush(self.heap, (priority, item))

    def dequeue(self):
        """Remove and return highest priority item."""
        if self.heap:
            return heapq.heappop(self.heap)[1]
        return None

    def display(self):
        """Display all elements."""
        print(self.heap)
# Sample usage
pq = PriorityQueue()
pq.enqueue("Task 1", 2)
pq.enqueue("Task 2", 1)
pq.enqueue("Task 3", 3)
print("Priority Queue contents:")
pq.display()  # Output: Priority Queue contents: [(1, 'Task 2'),
# (2, 'Task 1'), (3, 'Task 3')]
print("Dequeue:", pq.dequeue())  # Output: Dequeue: Task 2
print("Priority Queue after dequeue:")
pq.display()  # Output: Priority Queue after dequeue: [(2, 'Task 1'), (3, 'Task 3')]
#task8
```

Output:

```
Inserted Task A with priority 2
Inserted Task B with priority 1
Inserted Task C with priority 3
Queue contents: [(1, 'Task B'), (2, 'Task A'), (3, 'Task C')]
Dequeued: Task B
Queue contents: [(2, 'Task A'), (3, 'Task C')]
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding>
```

Observation:

1. The priority queue uses a heap structure where elements with higher priority (smaller number) are removed first.
2. The enqueue method inserts elements while maintaining heap order automatically.
3. The dequeue method efficiently removes the highest-priority element in logarithmic time.

Task:08

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque. Sample Input Code: class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Prompt:

For this implement the deque and do all the requriements Code:

```python
from collections import deque

class DequeDS:
    """
    Double-Ended Queue (Deque) implementation using collections.deque.
    Supports insertion and deletion from both front and rear.
    """

    def __init__(self):
        """Initialize an empty deque."""
        self.dq = deque()

    def insert_front(self, item):
        """
        Insert an element at the front of the deque.

        Parameters:
        item: The value to insert
        """
        self.dq.appendleft(item)
        print(f"{item} inserted at front")

    def insert_rear(self, item):
        """
```

DEBUG CONSOLE TERMINAL PROBLEMS 98 PORTS GITLENS SPELL CHECKER 21

```python
    def remove_front(self):
        Remove and return the front element.

        Returns:
        The removed element, or None if deque is empty
        """
        if not self.dq:
            print("Deque is empty")
            return None
        return self.dq.popleft()

    def remove_rear(self):
        """
        Remove and return the rear element.

        Returns:
        The removed element, or None if deque is empty
        """
        if not self.dq:
            print("Deque is empty")
            return None
        return self.dq.pop()

        return self.dq.pop()

    def display(self):
        """Display all elements in the deque."""
        print("Deque contents:", list(self.dq))


# -------- Sample Usage --------
d = DequeDS()

d.insert_front(10)
d.insert_rear(20)
d.insert_front(5)

d.display()

print("Removed from front:", d.remove_front())
print("Removed from rear:", d.remove_rear())

d.display()
```

DEBUG CONSOLE TERMINAL PROBLEMS 98 PORTS GITLENS SPELL CHECKER 21

Output:

```
10 inserted at front
20 inserted at rear
5 inserted at front
Deque contents: [5, 10, 20]
Removed from front: 5
Removed from rear: 20
Deque contents: [10]
```

Obsrvation:

A deque allows insertion and deletion from both the front and rear efficiently.

appendleft and popleft handle front operations, while append and pop handle rear operations.

This structure is useful for applications requiring flexible queue behavior, such as sliding window problems.

Task:09

Task Description #9 Real-Time Application Challenge – Choose the

Right Data Structure Scenario:

Your college wants to develop a Campus Resource Management System that

handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority Queue o

Linked List

o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

Prompt: for this implement the all the requirements of my document:

```python
#task09:
from collections import deque

class CafeteriaQueue:
    """
    Queue implementation for managing cafeteria orders.
    Follows FIFO (First In, First Out) principle.
    """

    def __init__(self):
        """Initialize an empty order queue."""
        self.queue = deque()

    def place_order(self, student_name):
        """
        Add a student's order to the queue.

        Parameters:
        student_name: Name of the student placing the order
        """
        self.queue.append(student_name)
        print(f"Order placed by {student_name}")
```

DEBUG CONSOLE TERMINAL PROBLEMS 105 PORTS GITLENS SPELL CHECKER 22

```python
        print(f"Order placed by {student_name}")

    def serve_order(self):
        """
        Serve the next student in the queue.

        Returns:
        Name of the served student, or None if empty
        """
        if not self.queue:
            print("No orders to serve")
            return None
        served = self.queue.popleft()
        print(f"Order served to {served}")
        return served

    def display_queue(self):
        """Display all pending orders."""
        print("Pending orders:", list(self.queue))


# -------- Sample Usage --------
cq = CafeteriaQueue()
```

DEBUG CONSOLE TERMINAL PROBLEMS 105 PORTS GITLENS SPELL CHECKER 22

```python
    def display_queue(self):
        """Display all pending orders."""
        print("Pending orders:", list(self.queue))


# -------- Sample Usage --------
cq = CafeteriaQueue()

cq.place_order("Alice")
cq.place_order("Bob")
cq.place_order("Charlie")

cq.display_queue()

cq.serve_order()
cq.display_queue()
```

Output:

```
/assginment11.1.py"
5 inserted at front
Deque contents: [5, 10, 20]
Removed from front: 5
Removed from rear: 20
Deque contents: [10]
Order placed by Alice
Order placed by Bob
Order placed by Charlie
```

Observation:

1. The cafeteria system follows FIFO order, ensuring students are served fairly based on arrival time.
2. The queue efficiently handles adding new orders and serving existing ones without reordering.

3. This structure is ideal for real-time service systems where order of processing must be preserved.